# Internet of Things (IoT) Laboratory

## Solutions for Assignments 1 to 10

**How to use this document**

- Each assignment has the question, a complete model answer and a short marking note.
- Students should replace the examples with their own devices, values and drawings. Answers below are samples, not the only correct answers.
- Assignment 10 was cut off in the question paper after "Smart Night Light". A complete Smart Bin mini-project is given as the model answer.

# Assignment 1: Identify 5 IoT Devices

**Question:** *Identify 5 IoT devices in your home, college, or a public space. Create a table listing each device, its primary sensor, its primary actuator, and one societal benefit.*

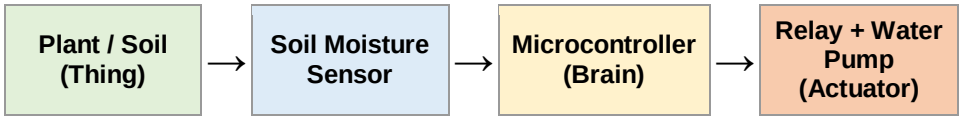
## Model answer

| No. | IoT Device | Primary Sensor | Primary Actuator | Societal Benefit |
|---|---|---|---|---|
| 1 | Smart thermostat / smart AC (home) | Temperature sensor | Compressor / fan relay | Saves electricity and reduces energy bills |
| 2 | Smart streetlight (public space) | Light sensor (LDR) and motion sensor | LED lamp driver | Saves public energy and improves road safety at night |
| 3 | Fitness band / smartwatch | Heart-rate (PPG) sensor and accelerometer | Vibration motor and display | Encourages healthy habits and early health awareness |
| 4 | Smart water tank controller (home/college) | Water level sensor | Water pump relay | Prevents water wastage and overflow |
| 5 | Smart attendance / CCTV camera (college) | Camera or RFID reader | Door lock or alarm buzzer | Improves campus security and safety |

# Assignment 2: Smart Plant Monitoring System

> **Question:** *Design a Smart Plant Monitoring System on paper. Draw a block diagram (plant, soil moisture sensor, microcontroller, water pump) and write a simple 3-step If-Then logic.*

## Block diagram




Water flows from the pump back to the plant, which completes the control loop. Power supply: 5 V adapter for the microcontroller, and a separate supply or battery for the pump, switched through the relay.

## Component roles

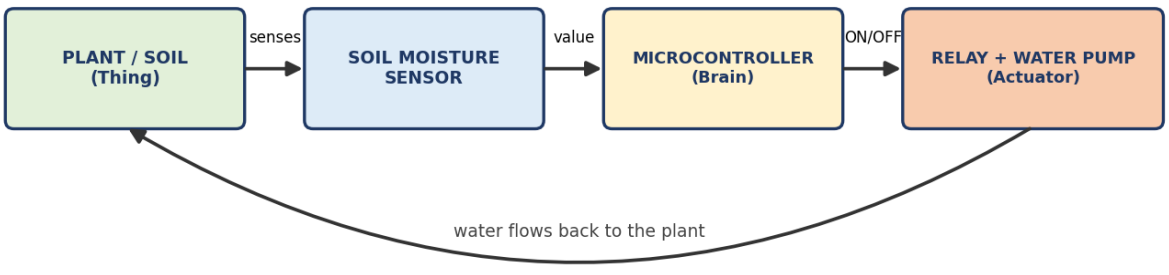
| Block | Role |
|---|---|
| Plant / Soil (thing) | The object being monitored. Its soil dryness is what we measure. |
| Soil moisture sensor | Gives an analog value that changes with the water content in the soil. Converted to a percentage. |
| Microcontroller (e.g. Arduino / ESP32) | Reads the sensor, compares with the threshold and decides what to do. |
| Relay + water pump (actuator) | Relay lets the low-power microcontroller pin switch the higher-power pump. |

## 3-step If-Then logic

1. Read the soil moisture value every 10 minutes and convert it to a percentage.
2. IF moisture < 30%, THEN turn the pump ON.
3. IF moisture >= 60%, THEN turn the pump OFF. Otherwise keep the current state and check again later.

*Note: Using two limits (30% and 60%) prevents the pump from switching ON and OFF rapidly near one value. This is called hysteresis.*

*IF moisture < 30%  THEN pump ON  |  IF moisture >= 60%  THEN pump OFF*



# Assignment 3: Publish-Subscribe Storyboard

> **Question:** *Choose one IoT communication model. Create a 4-panel comic strip or storyboard showing how a smart thermostat (publisher) sends temperature data to a user's phone (subscriber).*
>
> **AIM:** Show, in a 4-panel comic, how a thermostat sends temperature to a phone using Publish-Subscribe.

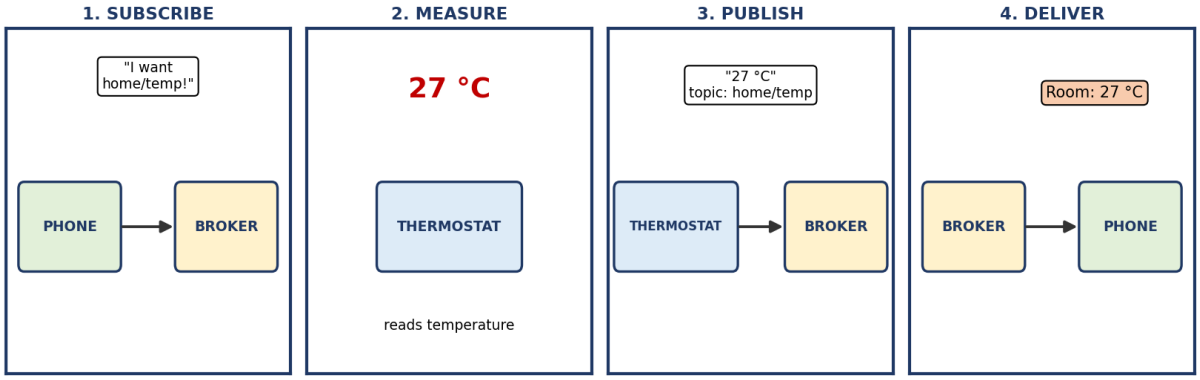
## Simple idea: YouTube example

A YouTuber (publisher) uploads a video. YouTube (broker) keeps it. People who pressed Subscribe (subscribers) get a notification. The YouTuber does not send the video to each person one by one. IoT works the same way.

| Role | In YouTube | In our smart home |
|---|---|---|
| Publisher | YouTuber | Smart thermostat |
| Broker | YouTube server | MQTT broker (cloud) |
| Subscriber | Viewer | User's phone app |
| Topic | Channel | home/livingroom/temp |

## Steps

1. Take a sheet and draw 4 equal squares.
2. Panel 1 (Subscribe): draw the phone telling the broker "I want updates from home/temp".
3. Panel 2 (Measure): draw the thermostat with a thermometer showing 27 °C.
4. Panel 3 (Publish): draw an arrow from the thermostat to the broker with the message "27 °C".
5. Panel 4 (Deliver): draw an arrow from the broker to the phone with a notification "Room: 27 °C".
6. Write one caption under every panel. Colour it neatly.




## Captions to write

1. Panel 1: The phone subscribes to the topic on the broker.
2. Panel 2: The thermostat measures the room temperature.
3. Panel 3: The thermostat publishes the reading to the broker.
4. Panel 4: The broker sends the reading to every subscriber. The phone shows it.

# Assignment 4: Python Temperature Sensor Simulation

> **Question:** *Write a Python script that simulates a temperature sensor by generating random numbers between 20 and 30 °C. Print each reading with a timestamp. Run it 10 times.*
>
> **AIM:** Write a Python program that pretends to be a temperature sensor (random 20 to 30 °C) and prints readings with a timestamp.

## Simple idea

We do not have a real sensor, so Python creates a random number between 20 and 30 and prints it like a sensor reading. The timestamp shows the time of each reading.

## What you need

5. Python 3 installed (or use any free online Python compiler).
6. A text editor or IDLE.

## Steps

1. Open IDLE or any editor and create a new file named sensor.py.
2. Type the program below exactly. Indentation (the spaces at the start of lines) is important in Python.
3. Save the file and run it. In IDLE press F5. In command prompt type: python sensor.py
4. The program prints 10 readings, one every second. If your teacher wants 10 separate runs, run the file 10 times.
5. Take a screenshot of the output for your journal.

## Program

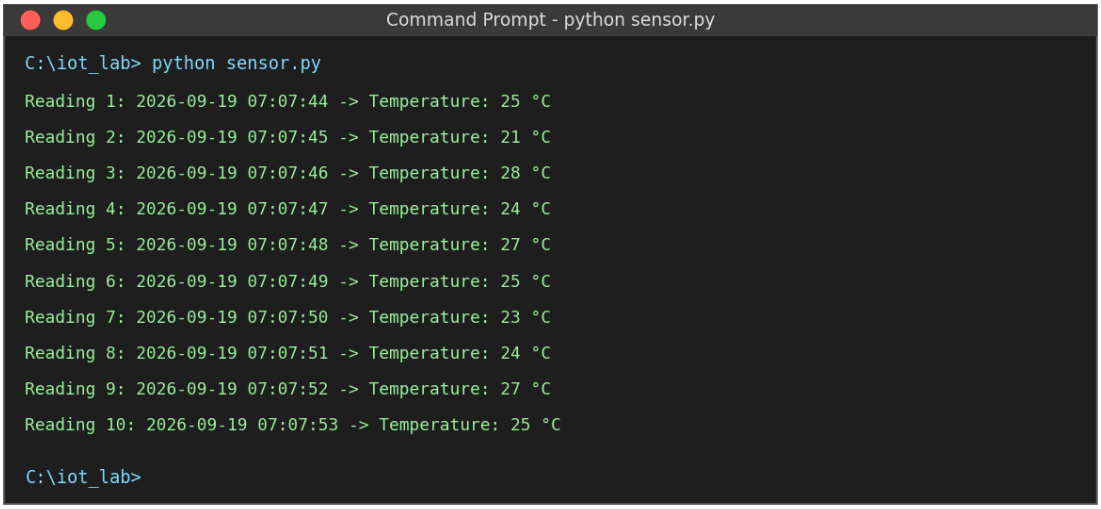
```
import random                           # to make random numbers
import time                             # to wait between readings
from datetime import datetime           # to get the current time

for i in range(1, 11):                  # repeat 10 times (1 to 10)
    temp = random.randint(20, 30)       # fake sensor reading, 20 to 30
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")   # timestamp
    print(f"Reading {i}: {now} -> Temperature: {temp} °C")
    time.sleep(1)                       # wait 1 second
```

## Sample output (from a real run)

```
Reading 1: 2026-09-19 07:07:44 -> Temperature: 25 °C
Reading 2: 2026-09-19 07:07:45 -> Temperature: 21 °C
Reading 3: 2026-09-19 07:07:46 -> Temperature: 28 °C
Reading 4: 2026-09-19 07:07:47 -> Temperature: 24 °C
Reading 5: 2026-09-19 07:07:48 -> Temperature: 27 °C
Reading 6: 2026-09-19 07:07:49 -> Temperature: 25 °C
Reading 7: 2026-09-19 07:07:50 -> Temperature: 23 °C
Reading 8: 2026-09-19 07:07:51 -> Temperature: 24 °C
Reading 9: 2026-09-19 07:07:52 -> Temperature: 27 °C
Reading 10: 2026-09-19 07:07:53 -> Temperature: 25 °C
```

## Screenshot example :Output




*Take a screenshot like this from your own computer and paste it in your journal.*

## Line-by-line meaning

| Line | What it does |
|---|---|
| import random | Loads the tool that makes random numbers. |
| for i in range(1, 11): | Starts a loop that runs 10 times. |
| random.randint(20, 30) | Picks a whole number from 20 to 30. This acts as the sensor. |
| datetime.now().strftime(...) | Gets the current date and time as text. |
| print(f"...") | Shows the reading on the screen. |
| time.sleep(1) | Pauses for one second, like a real sensor taking readings at intervals. |

# Assignment 5: MQTT vs HTTP Comparison

> **Question:** *Create a one-slide presentation or small poster comparing MQTT and HTTP. List 2 advantages and 2 disadvantages of each for a battery-powered sensor.*

> **AIM:** Make a one-slide poster comparing MQTT and HTTP for a battery-powered sensor: 2 advantages and 2 disadvantages of each.

## Simple idea

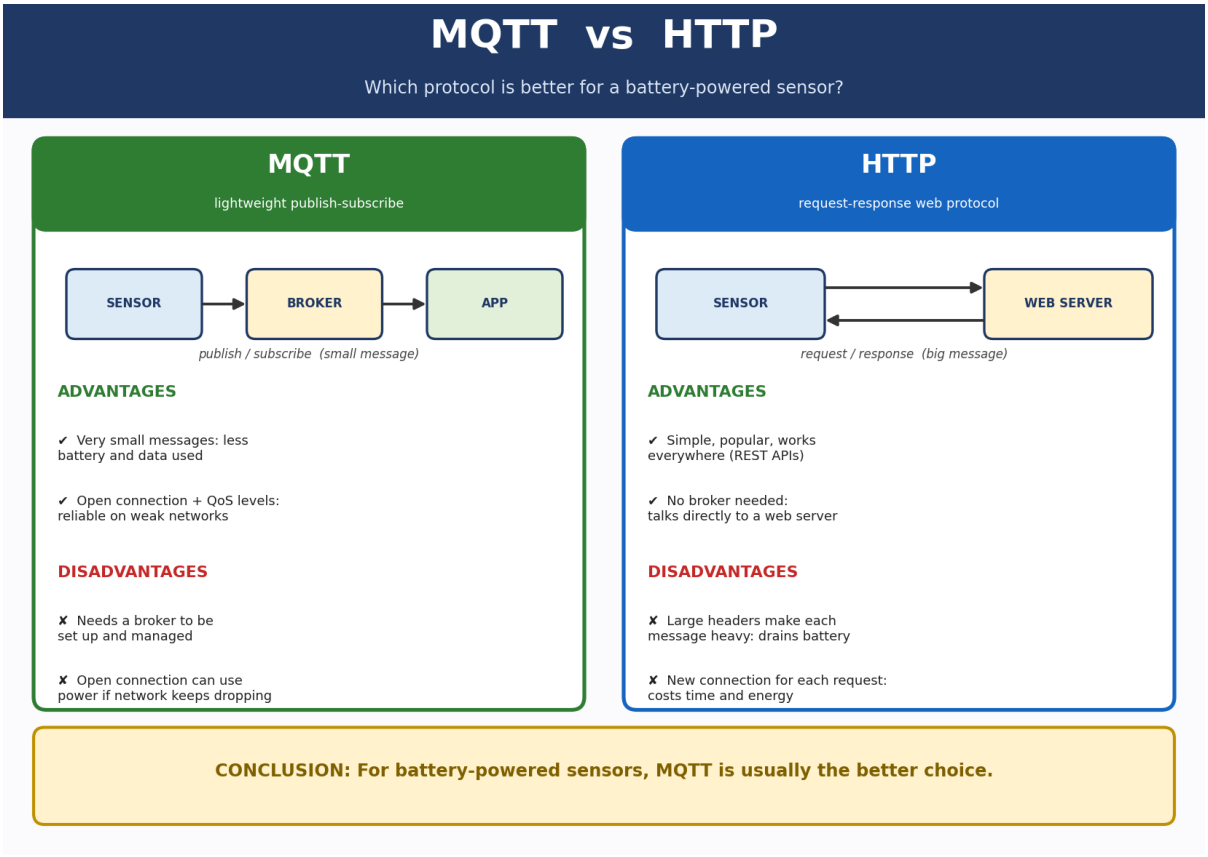
1.HTTP is like sending a full letter with a long address every time you want to say something. It is heavy.

2. MQTT is like sending a short SMS through a message centre. It is light, so a battery lasts longer.

## Poster / slide content

| | MQTT | HTTP |
|---|---|---|
| Type | Lightweight publish-subscribe protocol using a broker | Request-response protocol used by the web |
| Advantage 1 | Very small message header (as small as 2 bytes), so less data is sent and less battery is used | Universal and well understood. Works everywhere with simple tools and REST APIs |
| Advantage 2 | Keeps one connection open and supports QoS levels, so delivery is reliable even on weak networks | No broker needed. A sensor can send data directly to a web server or cloud |
| Disadvantage 1 | Needs a broker to be set up and maintained | Large headers make each message heavy, which drains battery |
| Disadvantage 2 | Keeping a long connection open can use power if the network is unstable | A new connection is opened for each request, which costs time and energy. It is also poor for pushing updates to the device |

## Ready-made poster (print it or redraw it on chart paper)



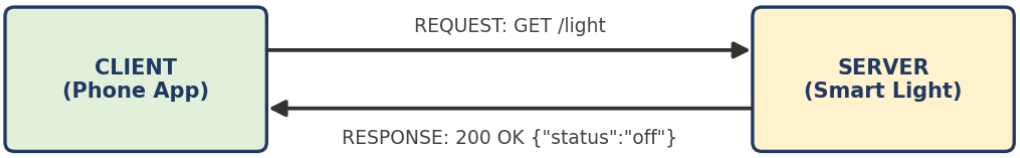

**Conclusion:** For a battery-powered sensor, MQTT is usually the better choice because it is lighter and uses less energy per message.

# Assignment 6: RESTful Light Switch on Paper

> **Question:** *Design a simple REST API for a smart light. Define two endpoints: GET /light and POST /light. Write down what a request and response would look like for each.*
>
> **AIM:** Write a REST API for a smart light with GET /light and POST /light, showing a request and response for each.

## Simple idea

REST is a way for a phone (client) to ask a device (server) for something using web addresses. GET means "tell me". POST means "do this". Every request gets a response with a status code (200 means OK) and data in JSON format.




## Steps

1. Write the base address of the light, for example smartlight.local.
2. Write the first endpoint: GET /light. This reads the status.
3. Write a sample request and a sample response for it.
4. Write the second endpoint: POST /light with body {"status": "on"}. This turns the light on.
5. Write a sample request and a sample response for it.
6. Make a summary table.

## Endpoint 1: GET /light

**Request:**

```
GET /light HTTP/1.1
Host: smartlight.local
```

**Response:**

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "status": "off" }
```

## Endpoint 2: POST /light

**Request:**

```
POST /light HTTP/1.1
Host: smartlight.local
Content-Type: application/json

{ "status": "on" }
```

**Response:**

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "status": "on", "message": "Light turned on" }
```

| Method | Endpoint | Body sent | Response |
|---|---|---|---|
| GET | /light | Nothing | 200 OK and {"status": "on"} or {"status": "off"} |
| POST | /light | {"status": "on"} | 200 OK and the new status. 400 Bad Request if the value is wrong |

# Assignment 7: Public IoT Cloud Platform Dashboard

**Question:** *Find a free, public IoT cloud platform demo (e.g. ThingSpeak or Ubidots). Interact with a pre-built dashboard and write a short paragraph on what data is shown and how it is presented.*

**AIM:** Open a free public IoT dashboard, look at it, and write a short paragraph on the data and how it is shown.

## Simple idea

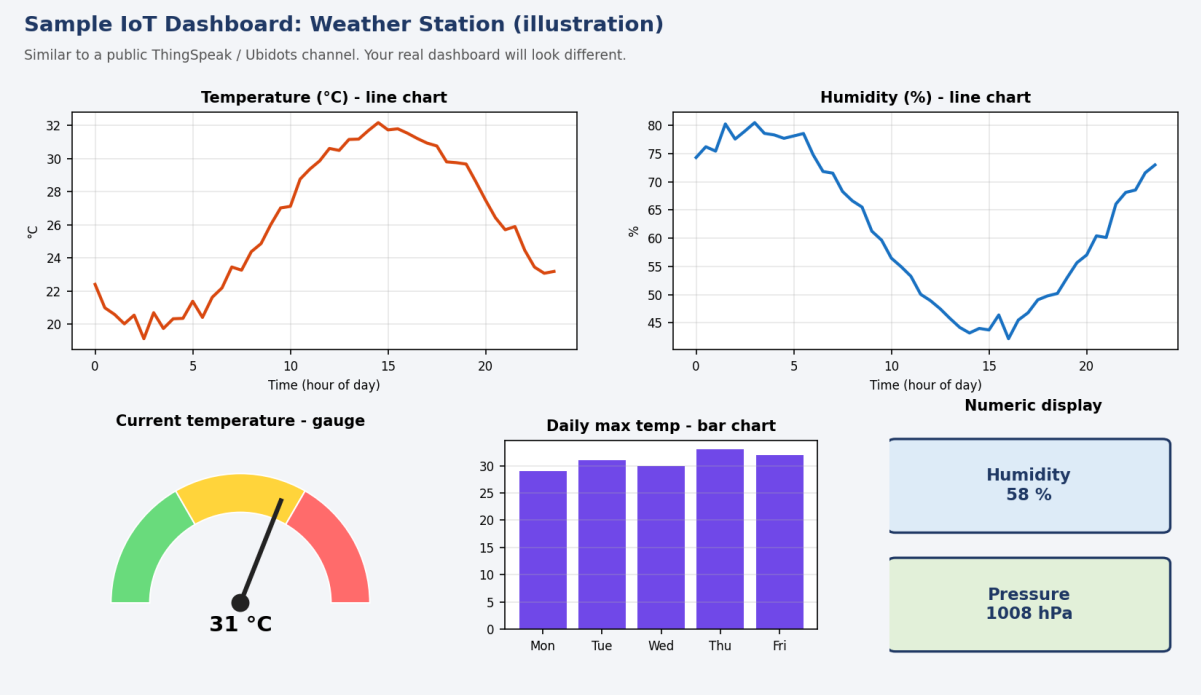
Sensors send data to the cloud. The cloud platform shows it as graphs, gauges and numbers on a dashboard, so anyone can understand it quickly.

## Steps

1. Open a web browser and go to ThingSpeak (thingspeak.com). You do not need an account just to look at public channels.
2. Find the Channels menu and choose Public Channels. Open any channel that has data, for example a weather station.
3. Look at each chart. Note the title, what is on the X axis (time) and the Y axis (value).
4. Try one interaction: change the time range or move the mouse over the graph to see values.
5. Take a screenshot of the dashboard.
6. Write your paragraph using the fill-in template below.

**TIP:** If ThingSpeak is slow or blocked in your college, use the demo dashboard of Ubidots or any other public IoT dashboard. The steps are the same.

## Example of what a dashboard looks like




*This is only an illustration to show the common parts (line chart, gauge, bar chart, numeric display). Your assignment needs a screenshot of a REAL public dashboard.*

## Model paragraph (ThingSpeak public channel)

I explored a public channel on ThingSpeak that shows weather station data. The dashboard displays temperature, humidity and atmospheric pressure sent by sensors at regular intervals. The temperature and humidity are shown as line charts against time, so we can see how they rise and fall during the day. The latest value is shown on a gauge and a numeric display, which makes the current condition easy to read at a glance. Each chart has a title, axis labels and a time range that can be changed. From this dashboard I understood that IoT platforms collect sensor data in the cloud, store it, and present it visually so users can monitor trends without reading raw numbers.

## Points to include (checklist)

- Name of the platform and the channel or dashboard opened.
- Which data was shown (for example temperature, humidity, location, status).
- How it was presented: line graph, gauge, bar chart, map, table or indicator light.
- One thing the student interacted with, such as changing the time range or hovering to see values.
- A screenshot of the dashboard.

# Assignment 8: Threat Model of a Video Doorbell

> **Question:** *List 2 potential vulnerabilities, 2 possible attackers, and 1 security requirement to mitigate each vulnerability.*
>
> **AIM:** List 2 vulnerabilities, 2 attackers, and 1 fix for each vulnerability.

## Simple idea

| Word | Meaning |
|---|---|
| Vulnerability | A weak point that can be attacked, like an unlocked window. |
| Attacker | The person who may misuse the weakness. |
| Security requirement | A rule or feature that closes the weakness, like adding a lock. |

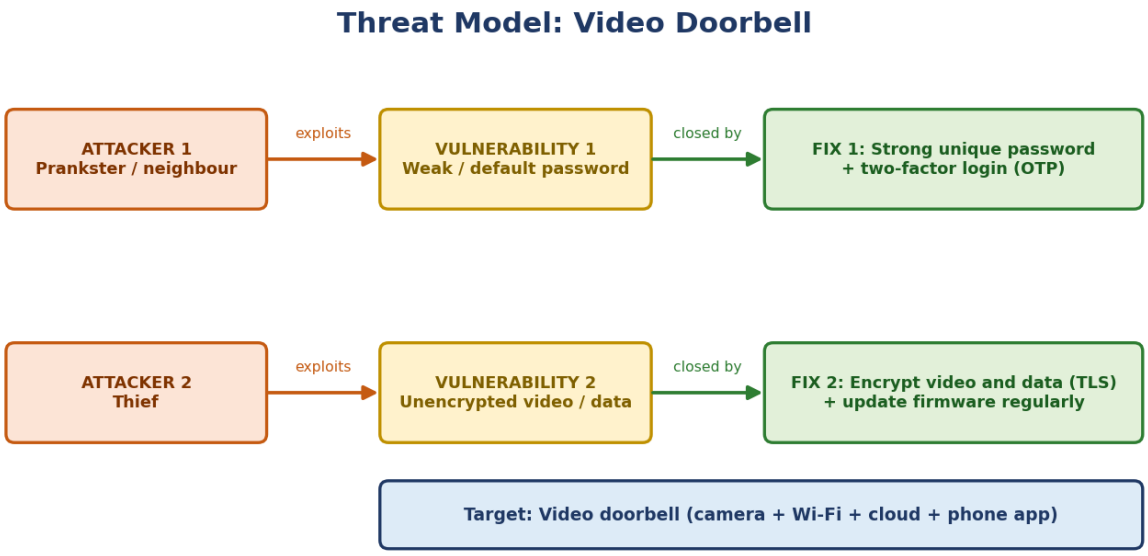
## Steps

1. Imagine a video doorbell: camera, Wi-Fi, cloud storage, phone app.
2. Ask: where can it be weak? Think of passwords, data travelling on Wi-Fi, old software.
3. Ask: who would attack? Think of a prankster, a thief, a stalker.
4. For every weakness write one fix.
5. Present it in a table.

## Final answer

| No. | Vulnerability | Possible attacker | Security requirement (fix) |
|---|---|---|---|
| 1 | Weak or default password on the account or Wi-Fi | Prankster who guesses the password and watches the camera | Force a strong, unique password and offer two-factor login (OTP). |
| 2 | Video and data sent without encryption | Thief who spies on the network to see when nobody is home | Encrypt video and data (HTTPS/TLS) when sending and storing. Update firmware regularly. |

## Threat model picture



# Assignment 9: M2M vs SCADA

> **Question:** *In a short paragraph, compare and contrast M2M and SCADA. Use one real-world example for each.*
>
> **AIM:** Compare M2M and SCADA in a short paragraph with one real example each.

## Simple idea

7. M2M (Machine-to-Machine): machines send data to each other or to a server automatically, without a person. Example: a vending machine tells the company when a product is finished.
8. SCADA (Supervisory Control and Data Acquisition): a control room where an operator watches an entire factory or plant on a screen and controls valves, pumps and motors. Example: a water treatment plant.

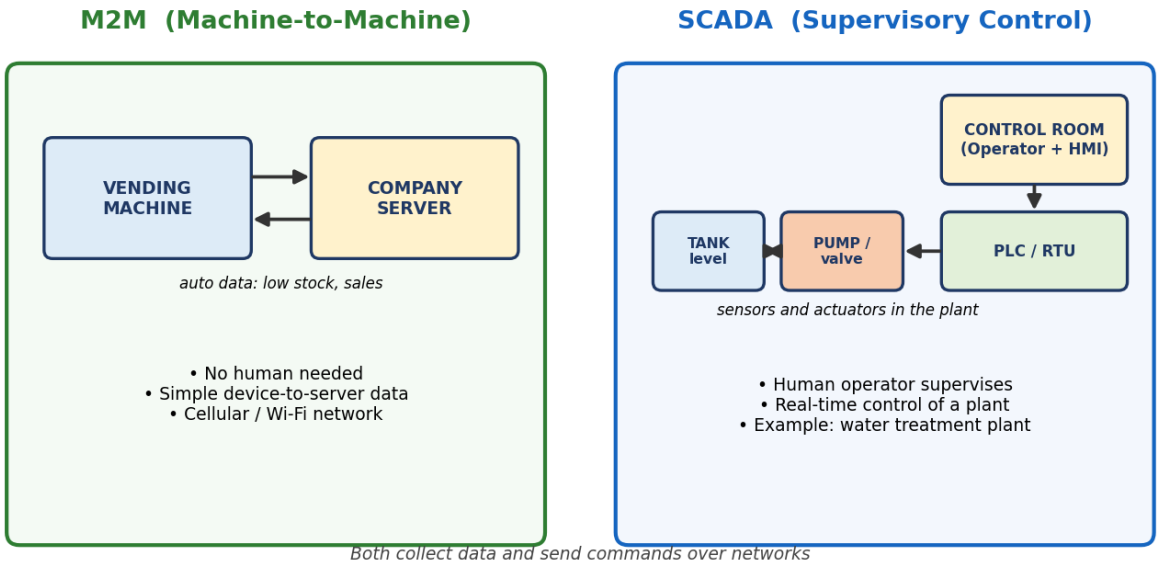
## Steps

1. Write one line defining M2M and give the vending machine example.
2. Write one line defining SCADA and give the water treatment plant example.
3. Write one similarity (both collect data and use networks).
4. Write two differences (purpose and human role).
5. Join all lines into one short paragraph.

## Final paragraph

M2M means machines exchange data automatically with each other or with a server with almost no human involvement. For example, a vending machine reports its sales and low stock to the company's server. SCADA is a system that lets an operator monitor and control an industrial process from a central control room using sensors, controllers and a screen. For example, in a water treatment plant the operator watches tank levels and controls pumps and valves. Both collect data and send commands over networks. However, M2M is simple device-to-device communication used in many commercial applications, while SCADA is designed for real-time supervision and control of a large industrial process by a human operator.

## Comparison picture



## comparison table

| Point | M2M | SCADA |
|---|---|---|
| Main purpose | Automatic data exchange between machines | Central monitoring and control of industrial processes |
| Human role | Very little | Operator supervises through an HMI |
| Scale | Small to large, often many scattered devices | Usually one plant or a large industrial site |
| Network | Cellular, Wi-Fi, LPWAN | Wired or dedicated industrial networks |
| Example | Vending machine reporting stock | Water treatment plant control room |

# Assignment 10: Final Mini-Project (Smart Bin)

> **Question:** *Choose a simple IoT system (e.g. Smart Bin that alerts when full, Smart Night Light). Describe the complete picture.*

## 1. Abstract

Overflowing dustbins spread dirt and bad smell, and fixed cleaning rounds waste fuel and time. This project builds a Smart Bin that measures how full it is using an ultrasonic sensor. An ESP32 microcontroller calculates the fill level and, when the bin is 80% full, turns on a buzzer and LED and sends an alert through Wi-Fi using the MQTT protocol to a cloud dashboard and the cleaner's phone. Cleaners then empty only the bins that are full. The system is low cost, easy to build and can be used in colleges, streets and hospitals.

## 2. Problem statement

a) Public dustbins overflow before they are emptied, causing dirt, smell, insects and health risk.

b) Cleaners follow a fixed route and visit bins that are still empty, wasting fuel, time and labour.

c) There is no data to know which bins fill fastest.

## 3. Objectives

1. To measure the fill level of a dustbin automatically.
2. To alert people locally (buzzer and LED) when the bin is nearly full.
3. To send the level to the cloud so cleaning staff can see it on a phone or dashboard.
4. To reduce cost and keep surroundings clean by emptying only full bins.

## 4. Simple idea

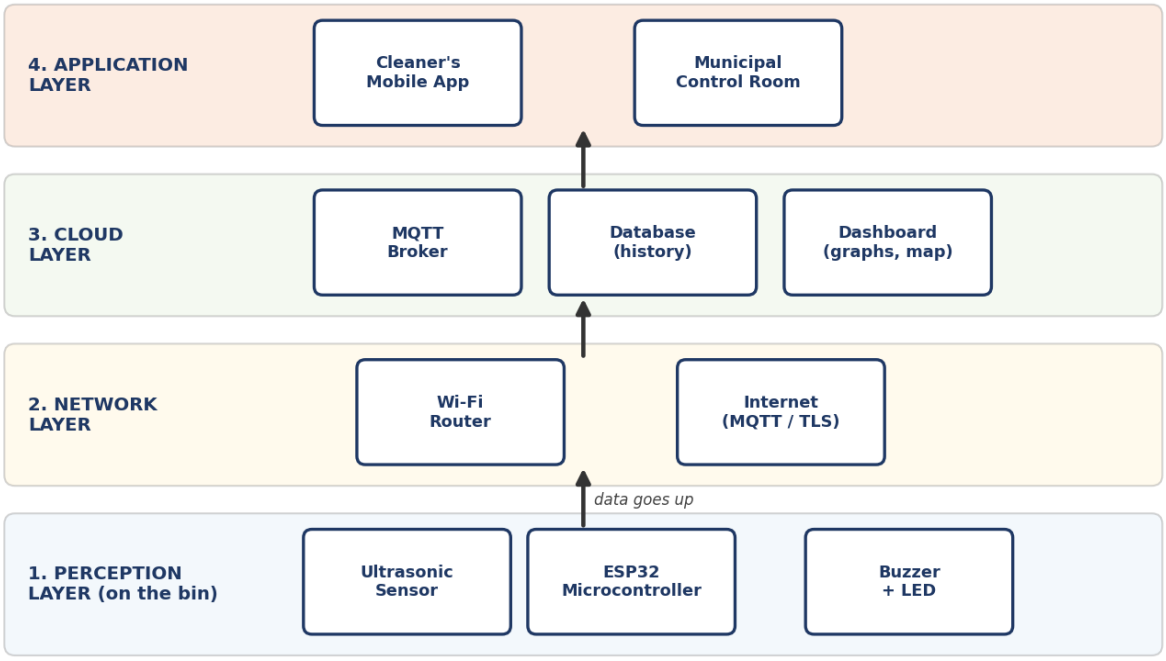
Think of the bin as a cup that we fill with water. An ultrasonic sensor at the top sends a sound wave down and listens for the echo. Less distance to the garbage means a fuller bin. The ESP32 turns the distance into a percentage. If it reaches 80%, it warns and reports. It follows the basic IoT chain:

**SENSE (ultrasonic) → THINK (ESP32) → ACT (buzzer + LED) → SHARE (Wi-Fi, MQTT, cloud, phone)**

## 5. Components and approximate cost

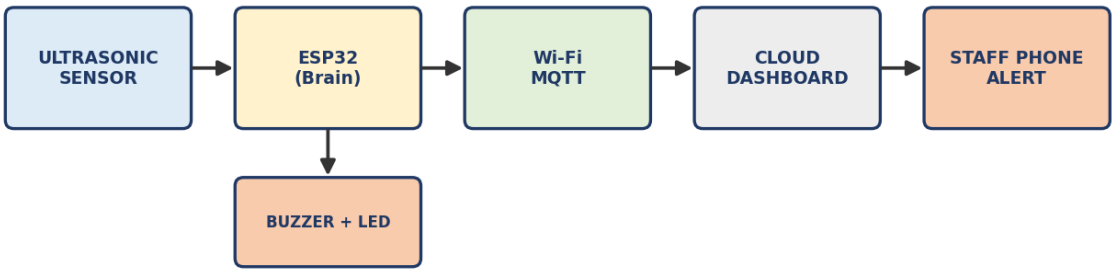
| Component | Purpose | Type | Cost (₹)* |
|---|---|---|---|
| ESP32 dev board | Brain with built-in Wi-Fi | Controller | 500 |
| HC-SR04 ultrasonic sensor | Measures distance to garbage | Sensor | 80 |
| Buzzer | Sound alert when full | Actuator | 20 |
| LED + 220 Ω resistor | Light alert when full | Actuator | 10 |
| 1 kΩ + 2 kΩ resistors | Voltage divider for the echo pin | Protection | 5 |
| Breadboard + jumper wires | Connections | Accessory | 140 |
| USB cable + 5 V adapter | Power supply | Power | 150 |
| Small dustbin / box | The bin (or a model) | Body | Existing |
| TOTAL | | | about 905 |

## 6. System architecture (4 layers)



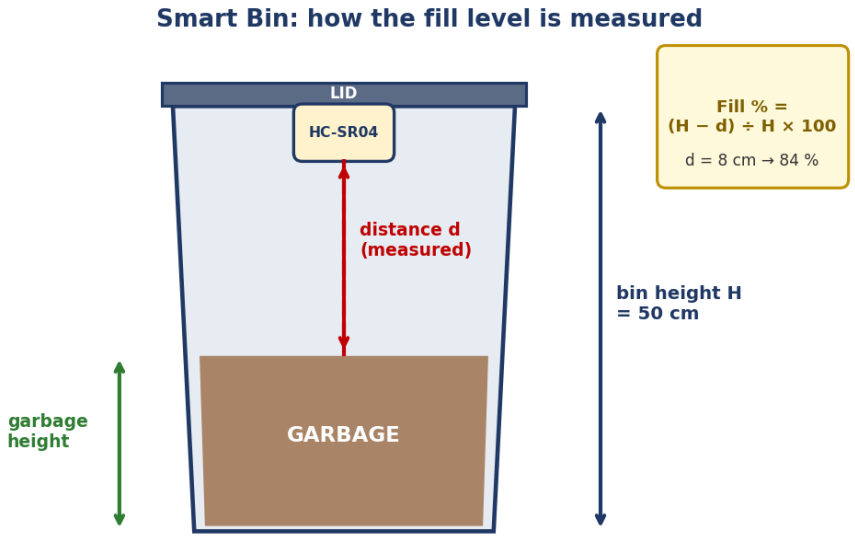

| Layer | What happens here |
|---|---|
| 1. Perception | Sensor measures the fill level, ESP32 calculates, buzzer and LED warn. |
| 2. Network | Wi-Fi router and the internet carry the MQTT messages. |
| 3. Cloud | MQTT broker receives messages, database stores history, dashboard shows graphs. |
| 4. Application | The cleaner's mobile app and the control room use the information. |

## 7. Block diagram (flow of data)




*The ESP32 also drives the buzzer and LED on the bin itself. The arrows show the direction of data.*

## 8. Physical model and fill-level calculation



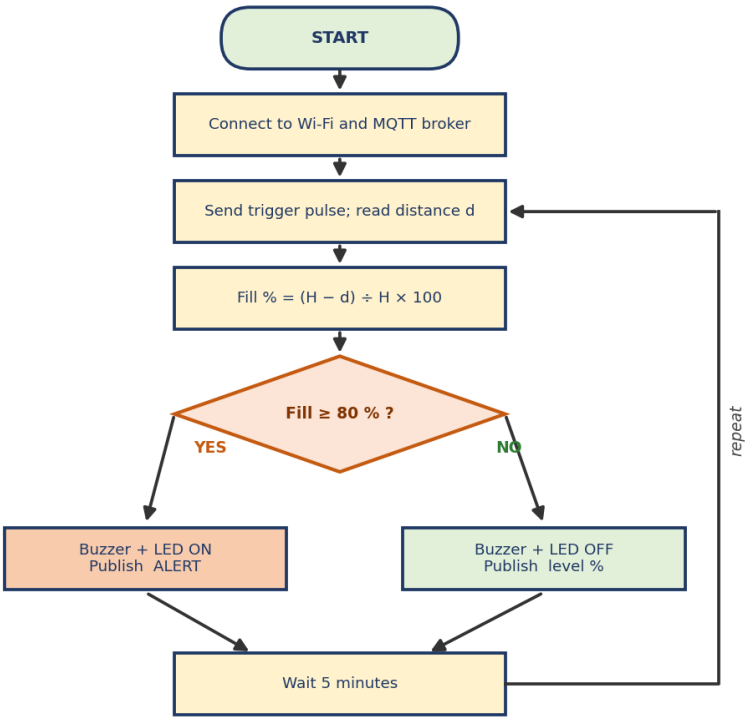

Fixed the sensor on the lid, facing down. Let H = bin height and d = distance measured by the sensor.

**Fill level (%) = (H − d) ÷ H × 100**

| Bin height H | Distance d | Calculation | Fill level |
|---|---|---|---|
| 50 cm | 50 cm | (50 − 50) ÷ 50 × 100 | 0 % (empty) |
| 50 cm | 25 cm | (50 − 25) ÷ 50 × 100 | 50 % (half) |
| 50 cm | 10 cm | (50 − 10) ÷ 50 × 100 | 80 % (ALERT limit) |
| 50 cm | 8 cm | (50 − 8) ÷ 50 × 100 | 84 % (ALERT) |

> **TIP:** How the sensor gets distance: distance = (echo time × 0.034 cm/µs) ÷ 2. We divide by 2 because the sound goes down and comes back.

## 9. Flowchart




## 10. Working, step by step

1. Power ON: the ESP32 connects to Wi-Fi and then to the MQTT broker.
2. The ESP32 sends a 10 microsecond pulse to TRIG. The sensor sends ultrasonic sound down.
3. The sound hits the garbage and comes back. ECHO stays HIGH for the travel time.
4. The ESP32 measures that time and converts it to distance d in cm.
5. It calculates Fill % = (H − d) ÷ H × 100.
6. If Fill % is 80 or more: buzzer and LED turn ON, and the message ALERT is published.
7. If Fill % is below 80: buzzer and LED are OFF, and the level (for example 45) is published.
8. The broker sends the message to all subscribers: dashboard and cleaner's phone.
9. The ESP32 waits (10 s for demo, 5 minutes in real use) and repeats from step 2.

## 11.Benefits to society

- Cleaner surroundings and better hygiene.
- Lower fuel and labour cost because only full bins are collected.
- Data on usage can help plan where to place more bins.